

The Knife

Manuale Tecnico

Laboratorio Interdisciplinare A

Alessia Gerti, Ginevra Sibilla,
Marco Marin, Stefano Marocco

Introduzione manuale

Il manuale tecnico nasce con l'obiettivo di descrivere le scelte progettuali, architettoniche e tecnologiche adottate nello sviluppo dell'applicazione.

La documentazione fornisce una panoramica sugli strumenti e sulle librerie esterne utilizzate; inoltre, illustra i principi che hanno guidato la realizzazione del software e, in particolare, i pattern di programmazione, alle strutture dati impiegate e alle modalità di serializzazione e gestione delle informazioni.

Dunque, il fine del manuale è chiarire la logica alla base del progetto, così da facilitare la comprensione e l'eventuale ripresa futura ed evidenziare la potenzialità di un approccio modulare e scalabile, che permette all'applicazione di crescere ed evolversi senza perdere coerenza interna.

Indice

Introduzione manuale.....	0
Installazione e setup ambiente di sviluppo.....	5
JDK - Java Developer Kit.....	5
Maven.....	8
Esecuzione.....	10
Scelte strutturali, algoritmiche e strutture dati.....	11
Pattern di programmazione utilizzate.....	11
Builder.....	11
Pattern MVC (Model-View-Controller).....	11
Pattern Composite.....	12
Modularità dell'applicazione.....	13
Gestione delle entità (Model).....	13
Espansione delle funzionalità tramite i Service.....	14
Personalizzazione della parte grafica (View).....	14
Vantaggi del pattern MVC nel progetto.....	14
Strutture dati.....	16
Strutture dati fondamentali.....	16
Serializzazione e strutture persistenti.....	16
Cenni e ottimizzazioni.....	17
Sistemi di salvataggio.....	19
Persistence Layer - Sistema di Gestione Dati.....	19
DataManager.....	19
JsonRepository<t>.....	19
RelationRepository.....	20
Model Layer - Dominio Applicativo.....	20
Service Layer - Logica di Business.....	20
Comunicazione tra Layer.....	21
Punti di forza architetturali.....	21
Librerie utilizzate.....	22
Limiti della soluzione sviluppata.....	24
Sitografia e bibliografia.....	25

Installazione e setup ambiente di sviluppo

Per poter runnare TheKnife è necessario avere:

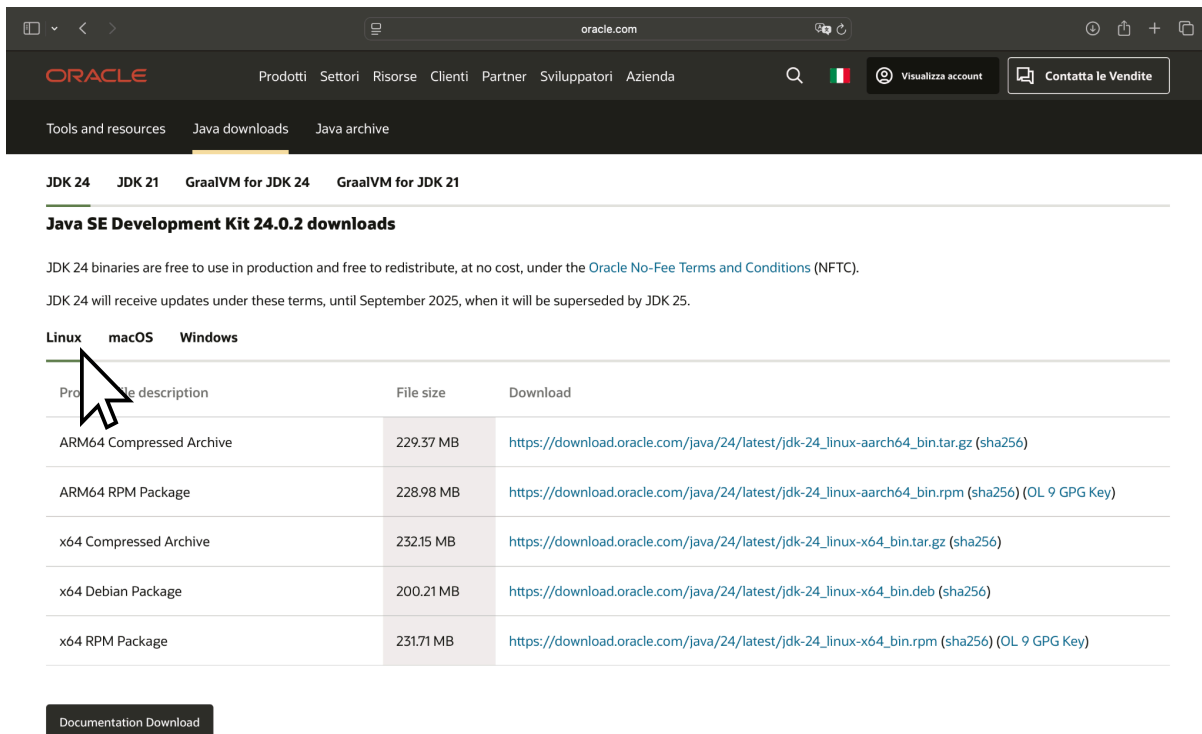
JDK - Java Developer Kit

Per installare il JDK è necessario collegarsi sul sito di Oracle e sotto la voce "Java Downloads" si troverà l'installazione che più si addice al sistema operativo del computer. Si consiglia di scaricare la versione più recente e aggiornata che si trova a disposizione (in questo momento, JDK 24.0.2).

Per la scelta del download da fare bisogna controllare l'architettura del proprio computer. Per venire a conoscenza di questa specifica, basterà cercarla su internet (es.: "architettura MacBook Air 2020"). Di seguito è riportata una lista del download da selezionare in base all'architettura del computer divisa per sistemi operativi.

Linux

(per le distribuzione arch based è presente nel pacchetto AUR):



The screenshot shows the Oracle Java Downloads page for JDK 24.0.2. The page has a dark header with the Oracle logo and navigation links. Below the header, there are tabs for "Tools and resources", "Java downloads", and "Java archive". The "Java downloads" tab is selected. Under this tab, there are links for "JDK 24", "JDK 21", "GraalVM for JDK 24", and "GraalVM for JDK 21". The "JDK 24" link is selected. Below this, there is a section titled "Java SE Development Kit 24.0.2 downloads". A paragraph states that JDK 24 binaries are free to use in production and free to redistribute, at no cost, under the Oracle No-Fee Terms and Conditions (NFTC). Another paragraph states that JDK 24 will receive updates under these terms, until September 2025, when it will be superseded by JDK 25. Below this, there are tabs for "Linux", "macOS", and "Windows". The "Linux" tab is selected. A table lists the download links for Linux. A mouse cursor is pointing at the "Linux" tab.

Product description	File size	Download
ARM64 Compressed Archive	229.37 MB	https://download.oracle.com/java/24/latest/jdk-24_linux-aarch64_bin.tar.gz (sha256)
ARM64 RPM Package	228.98 MB	https://download.oracle.com/java/24/latest/jdk-24_linux-aarch64_bin.rpm (sha256) (OL 9 GPG Key)
x64 Compressed Archive	232.15 MB	https://download.oracle.com/java/24/latest/jdk-24_linux-x64_bin.tar.gz (sha256)
x64 Debian Package	200.21 MB	https://download.oracle.com/java/24/latest/jdk-24_linux-x64_bin.deb (sha256)
x64 RPM Package	231.71 MB	https://download.oracle.com/java/24/latest/jdk-24_linux-x64_bin.rpm (sha256) (OL 9 GPG Key)

Documentation Download

MacOS

ORACLE Prodotti Settori Risorse Clienti Partner Sviluppatori Azienda

Tools and resources Java downloads Java archive

JDK 24 JDK 21 GraalVM for JDK 24 GraalVM for JDK 21

Java SE Development Kit 24.0.2 downloads

JDK 24 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions \(NFTC\)](#).
JDK 24 will receive updates under these terms, until September 2025, when it will be superseded by JDK 25.

Linux **macOS** Windows

Product/file description	File size	Download
ARM64 Compressed Archive	225.09 MB	https://download.oracle.com/java/24/latest/jdk-24_macos-aarch64_bin.tar.gz (sha256)
ARM64 DMG Installer	224.57 MB	https://download.oracle.com/java/24/latest/jdk-24_macos-aarch64_bin.dmg (sha256)
x64 Compressed Archive	228.05 MB	https://download.oracle.com/java/24/latest/jdk-24_macos-x64_bin.tar.gz (sha256)
x64 DMG Installer	227.51 MB	https://download.oracle.com/java/24/latest/jdk-24_macos-x64_bin.dmg (sha256)

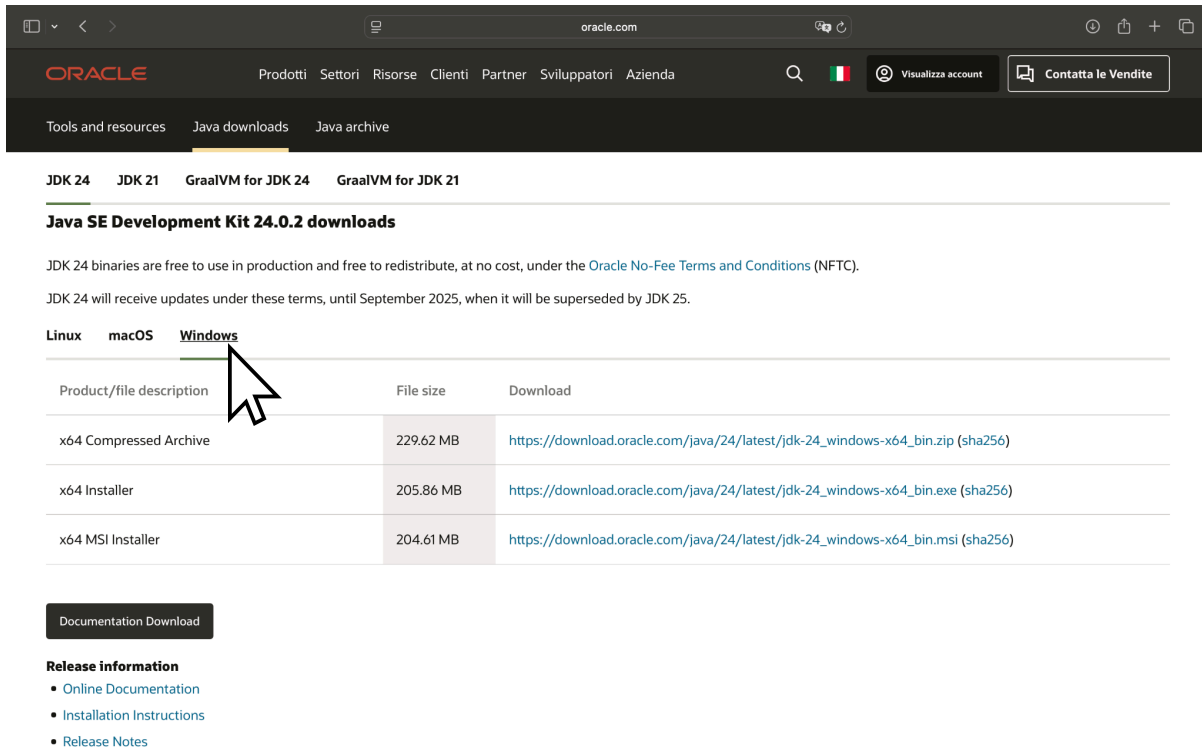
Documentation Download

Release information

- [Online Documentation](#)

- ARM64 DMG Installer: destinato a Mac con processori Apple Silicon (come M1, M2, ecc,...);
- x64 DMG Installer: per i Mac con processore Intel.

Windows



ORACLE Prodotti Settori Risorse Clienti Partner Sviluppatori Azienda

Tools and resources **Java downloads** Java archive

JDK 24 **JDK 21** GraalVM for JDK 24 GraalVM for JDK 21

Java SE Development Kit 24.0.2 downloads

JDK 24 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions](#) (NFTC).

JDK 24 will receive updates under these terms, until September 2025, when it will be superseded by JDK 25.

Linux macOS **Windows**

Product/file description	File size	Download
x64 Compressed Archive	229.62 MB	https://download.oracle.com/java/24/latest/jdk-24_windows-x64_bin.zip (sha256)
x64 Installer	205.86 MB	https://download.oracle.com/java/24/latest/jdk-24_windows-x64_bin.exe (sha256)
x64 MSI Installer	204.61 MB	https://download.oracle.com/java/24/latest/jdk-24_windows-x64_bin.msi (sha256)

[Documentation Download](#)

Release information

- [Online Documentation](#)
- [Installation Instructions](#)
- [Release Notes](#)

Dopo aver eseguito l'installazione del file corretto, si procede aprendo il file appena scaricato e seguendo le indicazioni riportate a schermo per concludere correttamente l'operazione.

Infine, per controllare che effettivamente il processo di installazione sia andato a buon fine, scrivere ed eseguire da terminale il comando "java -version". Se risulta stampato la versione del JDK scaricato in precedenza, l'installazione è avvenuta con successo.

In seguito all'installazione della JDK, si prosegue impostando le variabili d'ambiente PATH, per consentire l'esecuzione dei comandi Java. Anche in questo caso, per ogni sistema operativo le operazioni da eseguire sono differenti. Di seguito è riportata una lista con le istruzioni passo passo per impostare le suddette variabili.

Linux

```
sudo mkdir -p /usr/local/java
sudo tar -xzvf jdk-24_linux-x64_bin.tar.gz -C /usr/local/java/ se hai scaricato x64
Compressed Archive, mentre sudo tar -xzvf jdk-24_linux-aarch64_bin.tar.gz -C
/usr/local/java/ se invece hai scelto ARM64 Compressed Archive
sudo nano /etc/profile
export JAVA_HOME=/usr/local/java/jdk-24
export PATH=$PATH:$JAVA_HOME/bin
Per applicare le modifiche, eseguire source /etc/profile
```

MacOs

```
nano ~/.zshrc
export JAVA_HOME="/Library/Java/JavaVirtualMachines/jdk-24.jdk/Contents/Home"
export PATH="$PATH:$JAVA_HOME/bin"
Premere Command + O per salvare il file, Invio per confermare e Command + X per
uscire dall'editor
Per applicare immediatamente le modifiche alla sessione corrente del terminale
eseguire source ~/.zshrc
```

Windows

Inserire la variabile d'ambiente nel pannello dedicato (ricerca 'variabili d'ambiente' dallo start) e inserisci la variabile JAVA_HOME che punta al path della cartella estratta, alla cartella \bin (ad esempio: C:\Desktop\jdk-24\bin)

Maven

Per la gestione delle librerie, delle dipendenze e della creazione di jar ed eseguibili, è stato usato Maven, necessario per l'ambiente di sviluppo.

Linux

In base alla propria distribuzione installarlo dai repository ufficiali (apt o AUR che siano), qui riportato per le distribuzioni debian:

```
sudo apt install maven
```

MacOs

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew install maven
Per verificare l'installazione eseguire brew install maven
```

Windows

Downloading Apache Maven 3.9.11

Apache Maven 3.9.11 is the latest release: it is the recommended version for all users.

System Requirements

Java Development Kit (JDK)	Maven 3.9+ requires JDK 8 or above to execute. It still allows you to build against 1.3 and other JDK versions by using toolchains .
Memory	No minimum requirement
Disk	Approximately 10MB is required for the Maven installation itself. In addition to that, disk space will be used for your local Maven repository. The size of your local repository will vary depending on usage but expect at least 500MB.
Operating System	No minimum requirement. Start up scripts are included as shell scripts (tested on many Unix flavors) and Windows batch files.

Files

Maven is distributed in several formats for your convenience. Simply pick a ready-made binary distribution archive and follow the [installation instructions](#). Use a source archive if you intend to build Maven yourself.

In order to guard against corrupted downloads/installations, it is highly recommended to [verify the signature](#) of the release bundles against the public [KEYS](#) used by the Apache Maven developers.

	Link	Checksums	Signature
Binary tar.gz archive	apache-maven-3.9.11-bin.tar.gz	apache-maven-3.9.11-bin.tar.gz.sha512	apache-maven-3.9.11-bin.tar.gz.asc
Binary zip archive	apache-maven-3.9.11-bin.zip	apache-maven-3.9.11-bin.zip.sha512	apache-maven-3.9.11-bin.zip.asc
Source tar.gz archive	apache-maven-3.9.11-src.tar.gz	apache-maven-3.9.11-src.tar.gz.sha512	apache-maven-3.9.11-src.tar.gz.asc
Source zip archive	apache-maven-3.9.11-src.zip	apache-maven-3.9.11-src.zip.sha512	apache-maven-3.9.11-src.zip.asc

- 3.9.11 [Release Notes](#) and [Release Reference Documentation](#)
- latest source code from source repository
- Distributed under the [Apache License, version 2.0](#)
- other:
 - All current release sources (plugins, shared libraries,...) available at <https://downloads.apache.org/maven/>

Maven Daemon

Scaricare dal sito ufficiale di Maven il binary zip, estrarlo in una cartella a propria scelta, e - come fatto per il JDK - aggiungere una variabile d'ambiente:

Se non presente creare la variabile PATH, che punta al percorso dove è stata estratta la cartella di maven, della cartella \bin (c:\Desktop\apache-maven-3.9.6\bin)

Esecuzione

Una volta preparato tutto l'ambiente apri un terminale nella root del progetto (o attraverso un terminale o attraverso il tuo IDE o text-editor preferito)

Potrai eseguire il programma con l'utilizzo di maven grazie al comando:

```
mvn javafx:run
```

Se si volesse ribuildare il jar sarà possibile farlo con:

```
mvn clean package
```

Se si volesse ribuildare la documentazione sarà possibile farlo con:

```
mvn javadoc:javadoc
```

Se si volesse ribuildare l'exe si potrà fare grazie all'utilizzo di jpackage :

```
jpackage --type app-image \  
  --input [cartella_bin] \  
  --name [NomeApp] \  
  --main-jar [nome_jar_principale] \  
  --main-class [classe_principale] \  
  --module-path  
"[percorso_javafx_controls];[percorso_javafx_controls_win];[percorso_javafx_base];[p  
ercurso_javafx_base_win];[percorso_javafx_graphics];[percorso_javafx_graphics_win];[  
percorso_javafx_fxml];[percorso_javafx_fxml_win]" \  
  --add-modules javafx.controls,javafx.fxml
```

jpackage è uno strumento incluso nel JDK che permette di creare pacchetti di distribuzione per applicazioni Java. Consente di generare installer o immagini native (come .exe su Windows) che includono l'applicazione e tutte le dipendenze necessarie, facilitando l'installazione su sistemi degli utenti senza richiedere una configurazione manuale di Java.

Scelte strutturali, algoritmiche e strutture dati

Pattern di programmazione utilizzate

Builder

Il progetto fa uso del Builder Pattern, un design pattern creazionale che semplifica la costruzione di oggetti complessi. Invece di utilizzare costruttori con numerosi parametri (che possono risultare difficili da leggere e soggetti a errori), il Builder permette di costruire un oggetto passo dopo passo, invocando metodi dedicati per ciascuna proprietà e terminando con un metodo `build()`. Questo approccio ha reso il codice più leggibile e manutenibile, oltre a garantire maggiore flessibilità nel momento in cui fosse necessario modificare o estendere la struttura degli oggetti. È stato utilizzato nella creazione dei filtri di ricerca, in modo da poter applicare la logica di ogni singolo filtro e poi metterli in `and` nel metodo `build()`.

Pattern MVC (Model-View-Controller)

L'architettura dell'applicazione segue il pattern **Model-View-Controller (MVC)**, un paradigma molto diffuso nello sviluppo di applicazioni che prevedono un'interfaccia grafica interattiva. Questo pattern nasce dall'esigenza di separare in maniera chiara la gestione dei dati, la loro rappresentazione visiva e la logica di controllo, evitando che il codice risulti confuso e difficilmente manutenibile.

In particolare, il pattern MVC suddivide l'applicazione in tre componenti principali:

- **Model**

Il Model è responsabile della logica applicativa e della gestione dei dati. In esso vengono rappresentate le entità fondamentali del progetto, complete dei loro attributi, dei metodi di accesso e delle eventuali regole di coerenza. Nel nostro caso, i model sono stati progettati in modo semplice e chiaro, con l'obiettivo di definire gli oggetti che successivamente vengono serializzati o deserializzati tramite le librerie esterne (ad esempio Gson per la gestione del formato JSON). Queste classi rappresentano il "cuore" dell'applicazione, poiché descrivono gli elementi su cui operano i vari servizi.

- View

Il Model è responsabile della logica applicativa e della gestione dei dati. In esso vengono rappresentate le entità fondamentali del progetto, complete dei loro attributi, dei metodi di accesso e delle eventuali regole di coerenza. Nel nostro caso, i model sono stati progettati in modo semplice e chiaro, con l'obiettivo di definire gli oggetti che successivamente vengono serializzati o deserializzati tramite le librerie esterne (ad esempio Gson per la gestione del formato JSON). Queste classi rappresentano il "cuore" dell'applicazione, poiché descrivono gli elementi su cui operano i vari servizi.

- Controller

Il *Controller* rappresenta l'elemento di collegamento tra Model e View. In pratica, riceve gli input dell'utente dalla View, li interpreta e li traduce in operazioni sui Model, aggiornando di conseguenza la presentazione grafica. Nella nostra architettura, a ogni file `.fxml` corrisponde un file Controller dedicato (ad esempio `NomeViewController`), che si occupa di gestire i servizi associati all'entità rappresentata. Grazie a questa suddivisione, la logica di controllo è rimasta separata sia dalla definizione grafica che dalla gestione dei dati, semplificando il flusso complessivo dell'applicazione.

Pattern Composite

Il progetto fa uso anche del Composite Pattern, un design pattern strutturale che consente di trattare in maniera uniforme oggetti singoli e insiemi di oggetti. Questo pattern si rivela particolarmente utile quando è necessario rappresentare strutture gerarchiche, come alberi o insiemi nidificati, in cui un elemento complesso può essere composto da altri elementi dello stesso tipo.

L'idea alla base del Composite è definire un'interfaccia comune (ad esempio `Component`) che sia condivisa sia dagli oggetti elementari (`Leaf`) sia dai contenitori di oggetti (`Composite`). In questo modo, il client può interagire con qualsiasi istanza della gerarchia senza doversi preoccupare se si tratti di un singolo elemento o di una collezione di elementi.

L'uso del Composite ha portato due principali vantaggi al progetto:

- Uniformità di gestione: i metodi di manipolazione (ad esempio add, remove, getChild, oppure operazioni di calcolo e filtraggio) vengono invocati con la stessa interfaccia, sia su oggetti semplici che su strutture composte.
- Estendibilità e manutenibilità: l'aggiunta di nuovi tipi di nodi alla gerarchia non richiede modifiche al codice client, poiché essi rispettano comunque l'interfaccia comune.

Nel nostro caso, il Composite è stato applicato per organizzare e gestire strutture di dati gerarchiche legate ai filtri e ai criteri di ricerca. Ogni filtro elementare (ad esempio "filtra per nome", "filtra per data") rappresenta una foglia, mentre combinazioni più complesse di filtri (come filtri multipli con condizioni annidate) vengono rappresentate come nodi composite. Grazie a questo approccio, la logica dei filtri risulta modulare e scalabile, permettendo di costruire dinamicamente combinazioni anche molto articolate senza compromettere la leggibilità e la semplicità del codice.

Modularità dell'applicazione

Uno degli aspetti più significativi della nostra applicazione è l'elevato grado di **modularità**. L'architettura è stata progettata in modo tale da permettere l'aggiunta, la modifica o la rimozione di componenti senza dover intervenire in maniera massiccia sull'intero codice, riducendo i rischi di errori e semplificando la manutenzione futura.

- Gestione delle entità (Model)

Le entità sono state definite in classi modello indipendenti, ognuna con i propri attributi e metodi. Grazie a questa organizzazione, è possibile aggiungere nuove entità semplicemente creando una nuova classe Model. Le istanze di tali classi possono essere facilmente serializzate e deserializzate tramite le librerie Gson o JSON, garantendo che i nuovi dati vengano salvati e recuperati senza modificare il funzionamento delle entità già presenti. Allo stesso modo, rimuovere o modificare un Model non compromette l'intero sistema, poiché ciascun modello è incapsulato e gestito in maniera autonoma.

- **Espansione delle funzionalità tramite i Service**

Lo strato di *Service* svolge un ruolo cruciale nel rendere modulare l'applicazione. Ogni service espone i metodi necessari per operare sui dati del relativo Model: operazioni di creazione, lettura, aggiornamento o eliminazione (CRUD), oltre a eventuali logiche più complesse. Per introdurre nuove funzionalità, è sufficiente estendere o aggiungere un nuovo service, senza dover alterare la logica delle entità già esistenti. In questo modo, i servizi rappresentano un punto di espansione naturale dell'applicazione, che può crescere e adattarsi a nuove esigenze.

Le operazioni svolte da service sono tutte atomiche, in modo che se qualcosa va storto non venga intaccato il file system.

- **Personalizzazione della parte grafica (View)**

La modularità si riflette anche nell'interfaccia utente. Poiché la grafica è stata sviluppata tramite file `.fxml`, separati dalla logica applicativa, è possibile modificare, sostituire o arricchire una schermata intervenendo unicamente sulla View. Ad esempio, si possono aggiungere nuovi pulsanti, campi di input o tabelle senza intaccare la logica dei Controller o la gestione dei dati. Questo approccio facilita sia la manutenzione grafica sia l'eventuale restyling dell'interfaccia, che può essere adattata a nuove linee guida estetiche o a diverse esigenze di usabilità.

Vantaggi del pattern MVC nel progetto

L'adozione del pattern MVC ha offerto diversi vantaggi concreti:

- **Chiarezza del codice:** separando logica, grafica e dati, ogni file ha una responsabilità limitata e facilmente individuabile.
- **Manutenibilità:** eventuali modifiche a una parte del programma (es. aggiungere un attributo in un Model, cambiare lo stile grafico di una View) non impattano sugli altri livelli.
- **Collaborazione di gruppo:** i membri del team hanno potuto lavorare in parallelo, ad esempio uno sviluppando i controller e un altro curando le viste, senza creare conflitti.

- **Estendibilità:** l'architettura modulare rende più semplice l'aggiunta di nuove funzionalità, come la creazione di una nuova schermata con il relativo Controller e i metodi Service necessari.
- **Riutilizzabilità:** i componenti, in particolare i Service e i Model, possono essere riutilizzati in più contesti o adattati con modifiche minime.

Strutture dati

Viene implementato un approccio ibrido per la gestione dei dati, combinando strutture dati in memoria con persistenza su file JSON. Questa scelta architetturale riflette un compromesso tra semplicità di implementazione e performance, creando un sistema di persistenza leggero ma funzionale per applicazioni di medie dimensioni.

Strutture dati fondamentali

L'arrayList rappresenta la struttura dati principale utilizzata come cache in memoria per ogni JsonRepository

L'hashmap utilizzata, ad esempio, per il registry del repository, nel modello ristorante per gestire gli orari di apertura.

Utilizzando queste strutture viene implementata, in RelationRepository, una HashMap di arrayList, creando una struttura logica come la seguente

```
UserID_1 → [RistoranteID_A, RistoranteID_B, RistoranteID_C]
UserID_2 → [RistoranteID_B, RistoranteID_D]
```

vengono usate strutture date specializzate come gli enum o Optional<T> per i valori possibilmente assenti.

Serializzazione e strutture persistenti

Il sistema utilizza JSON come formato di serializzazione, creando una mappatura diretta tra oggetti Java e strutture dati persistenti:

```
{
  "id": "uuid-123",
  "username": "mario",
  "password": "$2a$12$...",
  "email": "mario@example.com",
  "ruolo": "UTENTE"
}
```

Viene implementato un TypeAdapter Pattern per LocalDateTime, con un GsonBuilder.

Cenni e ottimizzazioni

Il JsonRepository utilizza reflection per gestione generica dei tipi:

Java cancella i tipi generici a runtime (type erasure), rendendo impossibile deserializzare `List<T>` senza informazioni sul tipo, per questo è stato implementato un sistema di Reflection:

Serializzazione	<pre>//Nel costruttore di JsonRepository private final Type listType; public JsonRepository(String filePath, Class<T> entityClass) { //reflection: Crea Type parametrizzato per List<T> this.listType = com.google.gson.internal.\$Gson\$Types .newParameterizedTypeWithOwner(null, List.class, entityClass); }</pre>
Deserializzazione	<pre>private List<T> loadData() { try (Reader reader = new FileReader(file)) { //Usa il type Token per deserializzazione type-safe List<T> entities = gson.fromJson(reader, listType); return entities != null ? entities : new ArrayList<>(); } }</pre>

In questo modo: la chiave `Class<T>` garantisce che il repository sia del tipo corretto, solo il metodo specifico può inserire nel repository e il type token mantiene l'informazione di tipo a runtime.

Il calcolo delle distanze viene effettuato con Formula di Haversine.

Il DataManager implementa un pattern Composite per gestire collezioni eterogenee di repository: questa struttura permette accesso uniforme a repository di entità e relazioni attraverso interfacce diverse, ma semanticamente coerenti, ad esempio:

```
//Nel RistoranteService - Accesso semanticamente coerente
DataManager dataManager = DataManager.getInstance();

//Repository di entità
dataManager.registerEntityRepository(Ristorante.class, "data/ristoranti.json");
JsonRepository<Ristorante>
ristoranteRepo = dataManager.getRepository(Ristorante.class);

//Repository di relazioni
dataManager.registerRelationRepository("userLikesRistorante",
"data/userLikesRistorante.json");
```

Sistemi di salvataggio

Il sistema TheKnife adotta un'architettura a 4 layer principali:

PRESENTATION LAYER	Controllers JavaFX
SERVICE LAYER	Business Logic
PERSISTENCE LAYER	DataManager + Repositories
MODEL LAYER	Domain Entities

Persistence Layer – Sistema di Gestione Dati

DataManager

Il DataManager è il singleton centrale che orchestra tutto il sistema di persistenza:

Responsabilità:

- Repository factory: crea e gestisce repository JSON per ogni entità
- Relation manager: gestisce repository per relazioni N a N
- Punto di accesso unico: fornisce accesso centralizzato a tutti i repository creati

Pattern di Design:

- Singleton: una sola istanza globale
- Factory: crea repository su richiesta
- Registry: mantiene registri di repository attivi

JsonRepository<t>

Ogni entità del dominio ha il suo repository dedicato:

Caratteristiche Tecniche:

- Generics: type-safe per ogni entità (JsonRepository<Ristorante>)
- Cache Pattern: mantiene dati in memoria per performance
- JSON Serialization: utilizza Gson con adapter personalizzati
- CRUD Operations: create, Read, Update, Delete complete

Con questo layer il flusso di persistenza diventa così strutturato:

Entità → Cache in memoria → Serializzazione JSON → File system

RelationRepository

La struttura data scelta, come anticipato è in json, dunque la relazione n a n (che verrà utilizzato in DataManager) sarà così strutturata:

```
{
  "userId1": ["ristoranteId1", "ristoranteId2"],
  "userId2": ["ristoranteId3"]
}
```

La gestione delle relazione è data dalle operazioni di: aggiunta relazione, rimozione relazione, ricerca relazione, statistiche relazione (a scopo di debug)

Model Layer – Dominio Applicativo

Tutta la gestione di unicità all'interno del layer è affidata a BaseEntity, che garantisce il principio DRY (Don't Repeat Yourself), per fornire un ID univoco ad ogni entità.

È in questo livello che viene sviluppata la sicurezza con l'hashing delle password, gestito direttamente dal model User

A questo livello si sviluppa anche la classe FiltriDiRicerca, implementata attraverso il Builder Pattern per poter costruire query di filtri più complesse.

Come spiegato precedentemente il Builder Pattern garantisce:

- Fluent interface: sintassi leggibile
- Parametri opzionali: vengono applicati solo i filtri desiderati
- Immutabilità: oggetto finale immutabile

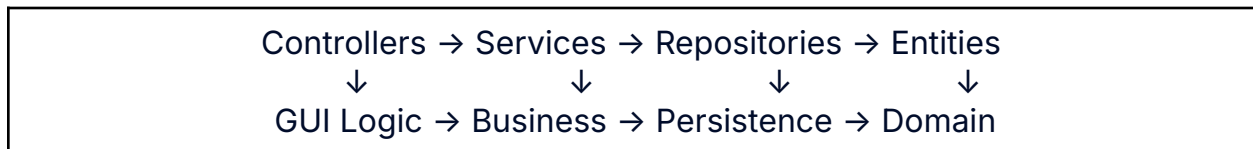
Service Layer – Logica di Business

Ad ogni entità principale viene associato un service dedicato, per la sua logica e i suoi metodi da esporre:

Gestendo attraverso le operazioni di CRUD principali (più metodi dedicati al singolo modello) otteniamo tutti i metodi che saranno utili, come anticipato nelle caratteristiche generali, questo permette una grande versatilità e modularità.

Comunicazione tra Layer

Dependency Flow



Punti di forza architetturali

Separazione responsabilità:

- Clear boundaries: ogni layer ha responsabilità specifiche;
- High cohesion: classi correlate raggruppate;
- Low coupling: dipendenze minimizzate via interfacce;

Estensibilità:

- Generic repositories: facile aggiunta di nuove entità;
- Service pattern: nuove business logic isolate;
- Builder pattern: query complesse senza explosion di parametri;

Manutenibilità:

- Domain rich: entità con business logic interna;
- Consistent patterns: stesso approccio per tutte le entità.

Librerie utilizzate

Nel progetto sono state impiegate alcune librerie esterne per semplificare lo sviluppo e rendere l'applicazione più robusta e funzionale:

- **JavaFX (controls, fxml)**

È il framework grafico scelto per la realizzazione dell'interfaccia utente.

- javafx-controls: mette a disposizione un'ampia gamma di componenti già pronti (pulsanti, tabelle, liste, campi di testo, ecc.), riducendo il tempo necessario per costruire interfacce complesse da zero.
- javafx-fxml: consente di definire l'interfaccia in file `.fxml`, separando così la logica applicativa dal layout grafico, secondo il paradigma MVC. Questo approccio migliora la leggibilità del codice e facilita eventuali modifiche o estensioni.

- **Atlantafx (v. 2.1.0)**

Libreria di stile per JavaFX che introduce un set di componenti e temi moderni ispirati al design "flat" tipico del web. È stata utilizzata per migliorare l'esperienza visiva, offrendo un'interfaccia più gradevole, coerente e facile da usare rispetto ai controlli standard di JavaFX.

- **JSON (v. 20240303)**

Fornisce strumenti semplici e leggeri per la creazione e manipolazione di stringhe e oggetti JSON. È stato utile soprattutto nei casi in cui si richiede la costruzione manuale di strutture dati JSON o l'accesso diretto agli attributi, come alternativa complementare a Gson.

- **Gson (v. 2.12.1)**

Libreria sviluppata da Google che consente la conversione automatica tra oggetti Java e formato JSON. Nel progetto è stata impiegata per la serializzazione (salvataggio di oggetti in formato testuale) e per la deserializzazione (ricostruzione di oggetti a partire da file o stringhe JSON). L'uso di Gson ha reso più immediata la gestione dei dati strutturati, evitando di scrivere codice di parsing manuale.

- **jBCrypt (v. 0.4)**

Implementa l'algoritmo BCrypt, comunemente utilizzato per l'hashing delle password. È stato integrato per garantire che le credenziali non vengano mai salvate in chiaro, ma in forma cifrata. Questa scelta incrementa la sicurezza del sistema, proteggendo gli utenti da eventuali accessi non autorizzati o furti di dati.

Limiti della soluzione sviluppata

Scalabilità

Nel `JsonRepository<T>`, ogni repository carica tutti i dati in memoria nella variabile cache dunque facendo una stima della memoria utilizzata:

10.000 ristoranti	~50MB RAM solo per i ristoranti
100.000 recensioni	~200MB RAM per le recensioni
1 milione di utenti	~100MB RAM per gli utenti
Totale: 350MB+ solo per i dati	350MB+ solo per i dati, senza contare JVM overhead

Memory Loading

Ogni ristorante viene caricato nella cache, dunque l'accesso è più veloce, ma l'avvio decisamente più lento

No Indexing

Tutte le ricerche (come find by Id) sono scansioni lineari, la complessità algoritmica sarà $O(n)$ ogni volta.

Possibilità di fix: usare hash index e B-tree per le ricerche

Concorrenza

Se si avviassero due istanze di TheKnife contemporaneamente si otterrebbero accessi concorrenti non gestiti nel file system, e si avrebbero dati inconsistenti

No Rollback

Non esistono meccanismi di rollback per annullare

Sitografia e bibliografia

IntelliJ IDEA

È stato l'ambiente di sviluppo utilizzato per realizzare il progetto. La scelta di IntelliJ ha permesso di beneficiare di supporto a Maven per la gestione delle dipendenze e strumenti integrati di debugging, che hanno reso più fluida ed efficiente la fase di sviluppo.

JavaDoc

Nel progetto, ogni classe, metodo e principali componenti del codice sono documentati con JavaDoc. Questa pratica facilita la manutenzione del codice, la lettura da parte di altri membri del gruppo e l'eventuale estensione futura del progetto.

Maven

Strumento di build e gestione delle dipendenze adottato per semplificare l'integrazione delle librerie esterne (es. Gson, jBCrypt, JavaFX, ecc.). L'uso di Maven ha garantito la portabilità e riproducibilità del progetto.

JavaFX

Framework grafico scelto per lo sviluppo dell'interfaccia utente. La documentazione e le guide disponibili hanno fornito supporto per capire la struttura dei controlli grafici, l'uso dei file FXML e l'organizzazione del codice.

AtlantaFX

È una libreria di stile e componenti grafici utilizzata insieme a JavaFX per rendere l'aspetto dell'UI più moderna.

Durante lo sviluppo, abbiamo usato i seguenti siti:

- *Maven* ↗
- *JavaFX* ↗
- *AtlantaFX* ↗